📘 Documentação do Micro Serviço: AI Gateway

1. Visão Geral

Este micro serviço atua como um **Gateway Centralizado de Inteligência Artificial**. Ele unifica o acesso a múltiplos provedores (OpenAI, Google Gemini, Anthropic, Groq, Ollama) através de uma única interface padronizada.

Principais Funcionalidades:

- **Roteamento Inteligente:** Troque de provedor sem alterar o código do cliente.
- **Auditoria Completa:** Logs detalhados de input, output, latência e erros.
- **Gestão de Custos:** Rastreamento de tokens e cálculo automático de preços (atualmente, o cálculo é feito preço/mil tokens).
- **Multi-tenant Lógico:** Separação de acesso por Sistemas e Consumidores via API Keys.

---

2. Autenticação

O sistema possui dois tipos distintos de autenticação:

A. Autenticação Administrativa (Dashboard)

Usada para configurar o sistema (criar rotas, usuários, ver logs).

- **Método:** Bearer Token (JWT).
- **Header:** `Authorization: Bearer <seu_token_admin>`
- **Obtenção:** Via rota `/admin/login`.

B. Autenticação de Consumo (Chat)

Usada por aplicações clientes (Frontends, Bots, Apps) para gerar texto.

- **Método:** API Key proprietária.
- **Header:** `x-consumer-key: <chave_gerada_no_dashboard>`

---

3. Guia de Implementação Rápida

Para começar a usar a IA, siga este fluxo:

1. **Login Admin:** Autentique-se na rota `/admin/login`.
2. O primeiro ADM é criado com as credenciais:

   name: `Super Admin`

   email: `admin@sistema.com`

   senha: `admin123`

3. **Criar Sistema:** Cadastre o sistema cliente (ex: "App Mobile").
4. **Criar Consumidor:** Cadastre o usuário/serviço (ex: "Chatbot Vendas"). **Guarde a `api_key` gerada.**
5. **Configurar Modelos:** Cadastre os modelos que deseja usar (ex: `gpt-4o`, `gemini-pro`). Os modelos devem ser salvos com o campo “name” exatamente igual a documentação de cada modelo. Exemplo: Gemini Flash seria cadastrado como gemini-2.0-flash
6. **Consumir:** Use a `api_key` do passo 3 para chamar a rota `/chat/`

---

4. Referência das Rotas (Endpoints)

🤖 Rota de Consumo (A Principal)

`POST /chat/`

Gera respostas de IA. Esta rota é **stateless** (não guarda histórico), o cliente deve enviar o contexto da conversa.

Headers:

- `x-consumer-key`: (Obrigatório) Chave do consumidor.

Body (JSON):

JSON
```
{
  "model": "gpt-4o",  // O 'alias' ou nome técnico do modelo cadastrado, o model é que deve ser enviado
  "system_instruction": "Você é um especialista em Python.",
  "messages": [
    { "role": "user", "content": "Olá, quem é você?" },
    { "role": "assistant", "content": "Sou uma IA." }, // Histórico (Contexto)
    { "role": "user", "content": "Explique o código acima." } // Pergunta atual
  ],
  "temperature": 0.7, // Criatividade (0.0 a 1.0)
```

```
  "max_tokens": 1000  // Limite de resposta (0 = automático)
}
```

Response (200 OK):

JSON
```
{
  "content": "O código acima demonstra uma função...",
  "tokens_used": 150,
  "input_tokens": 50,
  "output_tokens": 100,
  "total_cost": 0.0, // Custo calculado se configurado
  "model_used": "gpt-4o",
  "latency_ms": 450.5
}
```

⚙️ Rotas de Gerenciamento (Admin)

Modelos (`/models`)

Gerencia as rotas de IA disponíveis.

- `GET /models/`: Lista modelos ativos. Retorna nomes técnicos e aliases.
- `POST /models/`: Cria novo modelo.
    - *Feature:* Se `cost_input_per_1k` e `output` forem enviados como `0`, o sistema busca o preço automaticamente no catálogo global.

Payload:
JSON
```
{
  "name": "llama3",              // Nome técnico que o Ollama espera
  "alias": "Local Llama 3",       // Nome bonito para o Dashboard
  "provider_id": 1,              // ID do Provider "OpenAI" (ou um "Local" que use o driver
OpenAI)
  "api_key": "ollama",           // Ollama não exige chave real, mas o campo é obrigatório
  "url": "http://localhost:11434/v1", // <--- url com porta do ollama
  "cost_input_per_1k": 0,         // Local é de graça (ou custo de eletricidade)
  "cost_output_per_1k": 0,
  "auto_update_pricing": false    // Travamos o preço em 0 pois não está no LiteLLM
}
```

Sistemas e Consumidores (`/systems`, `/consumers`)

Gerencia quem pode acessar a API.

- `POST /systems/`: Cria um agrupador (ex: "Departamento Financeiro").
- `POST /consumers/`: Cria uma credencial de acesso.
    - *Retorno:* Inclui a `api_key` (única vez que ela é mostrada completa).

Logs e Auditoria (/logs)

Esta é a rota central de observabilidade do sistema. Ela permite consultar o histórico completo de interações, servindo tanto para **auditoria financeira** (contagem de tokens) quanto para **debugging técnico** (análise de prompts e respostas).

GET /logs/

Retorna uma lista paginada de registros de execução.

Autenticação:

- **Admin (Bearer Token):** Pode ver e filtrar logs de qualquer consumidor.
- **Consumer (API Key):** Vê **apenas** os seus próprios logs (Isolamento de Dados / Multi-tenant Lógico).

Parâmetros de Consulta (Query Params):

Parâmetro	Descrição
consumer_id	*(Opcional)* Filtra logs de um consumidor específico.
model_id	*(Opcional)* Filtra logs de um modelo de IA específico (ex: ver erros só do GPT-4).
transaction_uuid	*(Opcional)* Busca exata por um ID de transação (útil para rastrear um erro reportado pelo cliente).
limit	Quantidade de registros por página (Padrão: 50).
offset	Pular N registros (para paginação).

Exemplo de Resposta (JSON):

O grande diferencial deste endpoint é retornar os campos de contexto (input e output), permitindo "replay" da conversa.

JSON

```
[
  {
    "id": 105,
    "transaction_uuid": "a1b2c3d4-e5f6-...",
    "consumer_id": 12,
    "model_id": 5,
    "created_at": "2023-12-03T14:30:00.000Z",

    "http_status": 200,     // 200 = Sucesso, 500/429 = Erro
    "latency_ms": 450,      // Tempo de resposta da IA

    // Métricas de Consumo (Base para Faturamento)
    "tokens_input": 50,
    "tokens_output": 150,
    "tokens_total": 200,

    // O QUE FOI ENVIADO (Input Context)
    "input_context": {

        …

    },

    // O QUE FOI RECEBIDO (Output Context)
    "output_context": {

        …

      }
    }
  }
]
```

Cenários de Uso:

1. **Debug de Prompt:** "Por que a IA respondeu errado?" -> Inspecione o input_context para ver exatamente o que foi enviado.
2. **Análise de Falhas:** Se http_status for 500, o campo output_context conterá a mensagem de erro técnica (ex: "Rate Limit Exceeded").
3. **Performance:** Monitore o campo latency_ms para identificar modelos lentos.

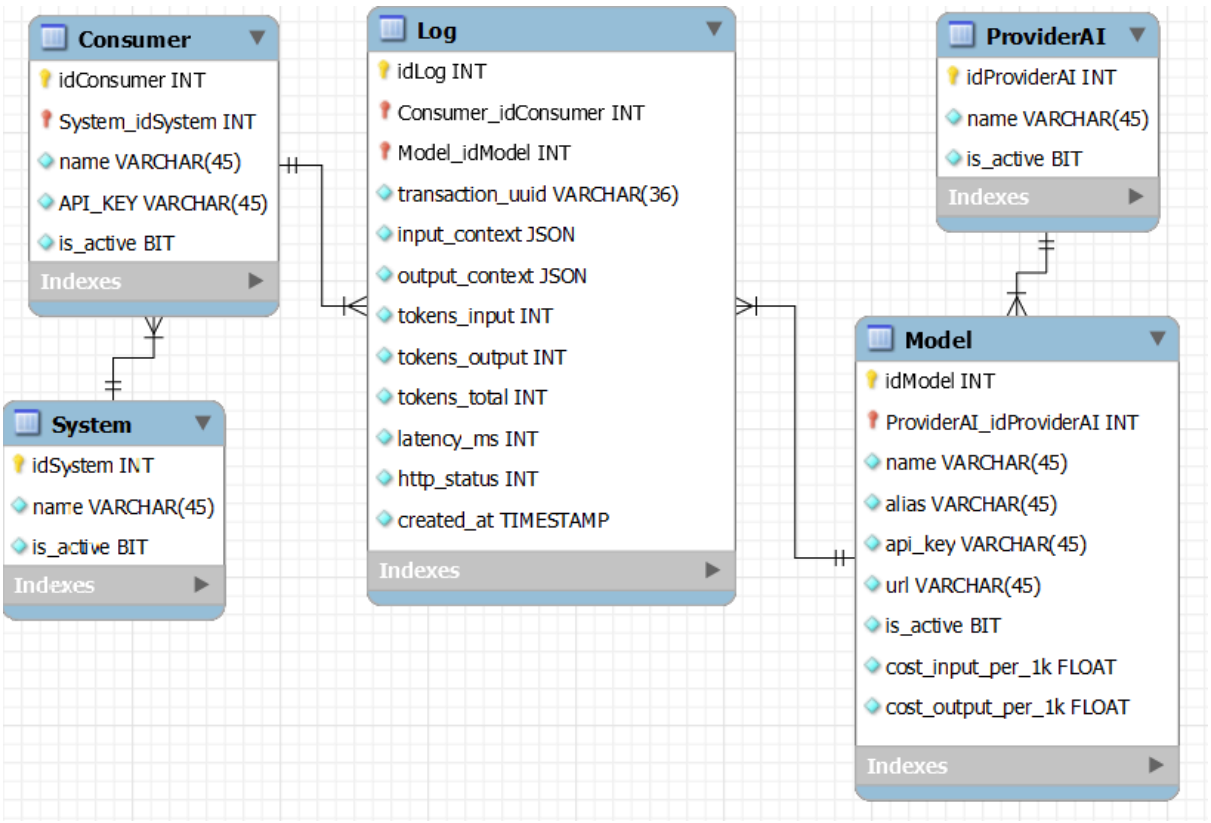
---

5. Suporte a Provedores

O sistema suporta nativamente os seguintes provedores (via Strategy Pattern):

1. **OpenAI:** (`gpt-4o`, `gpt-3.5-turbo`, etc.)
2. **Google Gemini:** (`gemini-1.5-flash`, `gemini-pro`)
3. **Anthropic:** (`claude-3-5-sonnet`)
4. **Compatíveis com OpenAI:**
    - **Groq:** Alta velocidade.
    - **Ollama:** Modelos locais (Llama 3, Mistral).
    - *Configuração:* Ao criar o modelo, preencha o campo `url` com o endpoint customizado (ex: `http://localhost:11434/v1` para Ollama). Caso aplicável (consultar documentação pydantic_ai).
5. **AWS:** Suporta 171 dependendo da região específica da aplicação aws. Exemplo: região “us-east-1” tem suporte para +30 IAs como:
    - **NVIDIA (**nvidia.nemotron-nano-9b-v2**)**
    - **Meta Llama(**meta.llama3-8b-instruct-v1:0**)**
    - **Amazon Titan(**amazon.titan-text-express-v1**)**
    - **Cohere(**cohere.command-text-v14)

6. MODELAGEM DO BANCO DE DADOS



(Modelo de banco centralizado, sem multi tenant)

7. CIAR AS CHAVES

AWS:Crie uma nova chave de acesso, que inclui uma nova secret access key.

- **Para criar uma nova secret access key para sua conta root, use a página de credenciais de segurança. Expanda a seção Chaves de acesso e clique em Criar nova root key.**
- **Para criar uma nova chave de acesso secreta para um usuário do IAM, abra o console do IAM. Clique em Usuário no painel Detalhes, clique no usuário IAM apropriado e, em seguida, clique em Criar Chave de Acesso na guia Credencial de segurança.**

NOTA: A chave de API utilizada pelo micro serviço é uma combinação de 3 chaves, na mesma string sem espaços e separadas por ponto e vírgula. Exemplo:

access_key_id;secret_acess_key;region_name

GOOGLE: É possível criar e gerenciar todas as suas chaves de API Gemini na página Chaves de API do Google AI Studio(https://aistudio.google.com/app/apikey?hl=pt-br).

OPENAI: Acesse platform.openai.com na aba a esquerda acesse api_keys clique em creat new secret key, essa é a chave que deve ser usada nos modelos com provedor openai.

8. PRÓXIMOS PASSOS SUGERIDOS

Manter contexto nas mensagens: Implementar uma Lista de mensagens antigas para que sirva de contexto para a próxima requisição, deve ser possível alterar a quantidade de mensagens antigas para manter o controle de tokens gastos a cada requisição. Pode ser implementado com message_history nativo do pydantic.

Ampliar tipos de dados de entrada: Entrada de Imagem, Audio, Vídeo e Documentos nas requisições.(Nem todos os modelos de IA suportam isso)

Reforçar o tratamento de erros: Como thottling por exemplo, atualmente ficam na pasta ai_core.